

DOCUMENTO FORMAL DE ARQUITECTURA

Backend API RESTful

TechCup Football

Equipo: Zeus-Codensa

Proyecto Academico

Version 1.0

Abril de 2026

Tabla de Contenidos

1. Introduccion
2. Descripcion General
3. Arquitectura del Sistema
 - 3.1 Diagrama de Componentes
 - 3.2 Descripcion de Capas
4. Modelo de Datos
 - 4.1 Diagrama E-R
 - 4.2 Descripcion de Entidades
5. Flujos Operacionales
6. Decisiones de Diseno
7. Stack Tecnologico
8. Conclusiones

1. Introduccion

El presente documento describe la arquitectura del backend RESTful de TechCup Football, una plataforma integral para la gestion de torneos de futbol. Este documento establece los fundamentos tecnicos, decisiones arquitectonicas y patrones implementados durante el desarrollo del sistema.

El proposito es proporcionar una referencia clara y completa para desarrolladores, arquitectos y stakeholders sobre la estructura, componentes, flujos de datos y decisiones de diseno del backend.

2. Descripcion General

TechCup Football es una plataforma web desarrollada bajo un modelo de arquitectura multicapa basada en Spring Boot 3.3.4 con Java 21. El backend implementa un servicio RESTful completo que gestiona:

- * Autenticacion y autorizacion mediante JWT y OAuth2
- * Gestion integral de torneos, equipos y jugadores
- * Persistencia de datos en PostgreSQL 15
- * Despliegue containerizado con Docker
- * Pruebas unitarias con cobertura del 85% mediante JaCoCo

3. Arquitectura del Sistema

3.1 Diagrama de Componentes

La arquitectura del sistema sigue un patron de capas bien definido:

CAPA	DESCRIPCION
Controller	Expone endpoints REST, valida solicitudes y coordina llamadas a servicios
Service	Logica de negocio, validaciones y orquestacion
Repository	Abstrae la persistencia mediante JPA
Security	Autenticacion JWT, OAuth2 y control de acceso
Infrastructure	PostgreSQL, Docker, despliegue

3.2 Descripcion de Capas

Controller: Recibe solicitudes HTTP, mapea parametros y delega logica a servicios mediante anotaciones Spring como @RestController y @RequestMapping.

Service: Contiene logica de negocio central, coordina multiples repositorios y aplica validaciones. Esta aislada de detalles de persistencia.

Repository: Abstrae operaciones de persistencia mediante Spring Data JPA con interfaces heredadas de JpaRepository.

Security: Implementa autenticacion JWT, OAuth2 y control de acceso basado en roles (ADMIN, ORGANIZER, PLAYER).

Infrastructure: PostgreSQL, Docker y configuracion de despliegue en multiples ambientes.

4. Modelo de Datos

4.1 Diagrama Entidad-Relacion

El modelo esta normalizado en tercera forma normal (3FN) y estructura las siguientes entidades principales:

ENTIDAD	ATRIBUTOS CLAVE	RELACIONES
User	id, email, password, role	1:N Player
Tournament	id, name, status, dates	N:M Team, 1:N Match
Team	id, name, city, coach	N:M Tournament, 1:N Player
Player	id, name, position, number	N:1 Team
Match	id, date, status, goals	N:1 Tournament
Invitation	id, status, dates	N:1 Player, N:1 Team

4.2 Descripcion de Entidades

User: Entidad raiz que representa un usuario. Almacena credenciales (email, password hasheado con bcrypt) y rol asignado (ADMIN, ORGANIZER, PLAYER).

Tournament: Representa un torneo con estado de ciclo de vida (DRAFT, ACTIVE, COMPLETED, CANCELLED). Mantiene relacion N:M con equipos participantes.

Team: Agrupa jugadores. Puede participar en multiples torneos. Mantiene lista de jugadores activos.

Player: Jugador con posicion, numero de dorsal y datos personales. Puede estar invitado a multiples equipos.

Match: Registra un encuentro entre dos equipos con resultado, fecha y estado (SCHEDULED, PLAYED, CANCELLED).

Invitation: Implementa la mecanica de invitaciones de jugadores a equipos. Estados: PENDING, ACCEPTED, REJECTED.

5. Flujos Operacionales

5.1 Autenticacion y Autorizacion

1. Usuario envia credenciales a POST /auth/login
2. Servicio valida contra User en base de datos
3. Se genera JWT firmado (ExpirationTime: 1 hora)
4. Token se retorna al cliente
5. Cliente incluye token en header: Authorization: Bearer <token>

Spring Security valida firma y expiracion en cada request protegido. Acceso por roles se controla mediante @PreAuthorize.

5.2 Gestion de Torneos

1. Organizador crea torneo con POST /tournaments
2. Sistema persiste en BD con estado DRAFT
3. Organizador anade equipos (POST /tournaments/{id}/teams)
4. Sistema genera calendario de partidos automatico
5. Transicion a estado ACTIVE permite actualizar resultados

5.3 Gestion de Usuarios

1. Registro: POST /auth/register con email, password, role
2. Validacion de email unico y contrasena fuerte
3. Contraseña hasheada con bcrypt antes de almacenar
4. Usuario creado con rol PLAYER por defecto

6. Decisiones de Diseño

6.1 Patrones de Diseño Implementados

Factory Method: Encapsula creación de entidades complejas (Tournament, Match, Invitation).

Builder Pattern: Simplifica construcción de DTOs y entidades con múltiples atributos opcionales.

Strategy Pattern: Validadores de estado (TournamentValidator, MatchValidator) con diferentes algoritmos.

Singleton Pattern: Servicios Spring funcionan como singletons para consistencia.

Dependency Injection: Spring inyecta dependencias automáticamente, facilitando testing.

6.2 Seguridad

JWT: Autenticación stateless sin sesiones server-side. Token contiene claims firmados. Expiración: 1 hora.

Bcrypt: Hashing de contraseñas con salt automático (cost factor: 10). Previene ataques de fuerza bruta.

RBAC: Tres roles (ADMIN, ORGANIZER, PLAYER) con control granular vía @PreAuthorize.

HTTPS y CORS: Obligatorio en producción. CORS solo de orígenes confiables.

6.3 Manejo de Errores

Excepciones Personalizadas: ResourceNotFoundException (404), ValidationException (400), UnauthorizedException (401).

Global Exception Handler: @RestControllerAdvice intercepta excepciones. Respuesta JSON consistente.

Logging Estructurado: SLF4J + Logback con niveles DEBUG, INFO, WARN, ERROR.

Validación de Entrada: Bean Validation (@Valid, @NotNull, @Email) previene inyección de datos.

7. Stack Tecnológico

COMPONENTE	DESCRIPCION
Runtime	Java 21 con features modernas
Framework	Spring Boot 3.3.4
Base Datos	PostgreSQL 15
ORM	Spring Data JPA + Hibernate 6
Seguridad	Spring Security 6, JWT, OAuth2, bcrypt
Documentación	Springdoc OpenAPI + Swagger UI
Testing	JUnit 5, Mockito, JaCoCo (85% cobertura)
Containerización	Docker + Docker Compose

8. Conclusiones

La arquitectura del backend de TechCup Football establece una base solida y escalable. La adopcion de patrones de diseno, arquitectura multicapa clara y practicas de seguridad robustas aseguran:

- * Mantenibilidad a traves de separacion clara de responsabilidades
- * Seguridad mediante autenticacion JWT + OAuth2 y control de acceso granular
- * Escalabilidad con modelo de capas desacoplado
- * Testabilidad con cobertura del 85% e inyeccion de dependencias
- * Observabilidad mediante logging estructurado y Swagger UI

El stack tecnologico elegido proporciona equilibrio entre robustez academica y aplicabilidad industrial.

Equipo Zeus-Codensa - Abril 2026